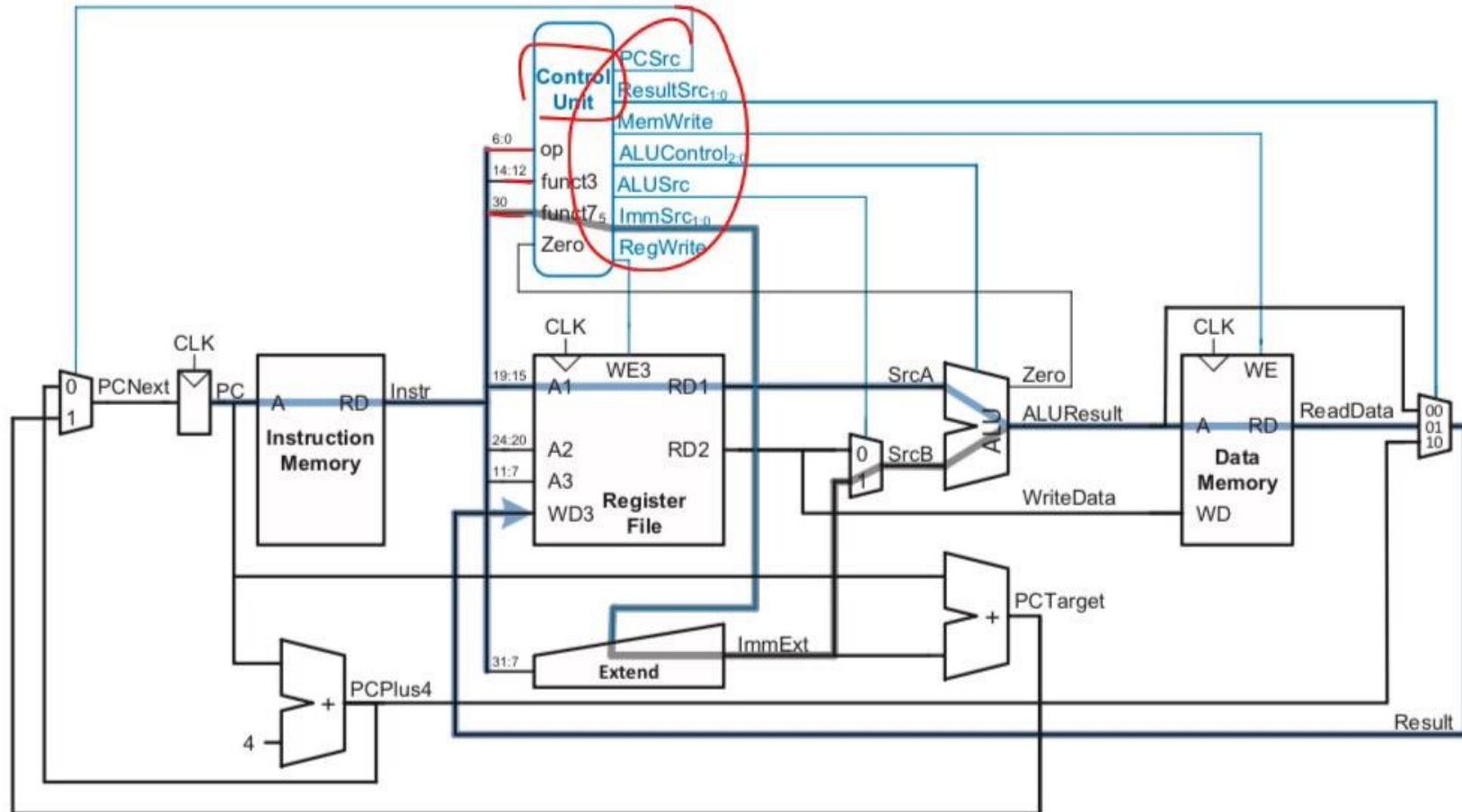




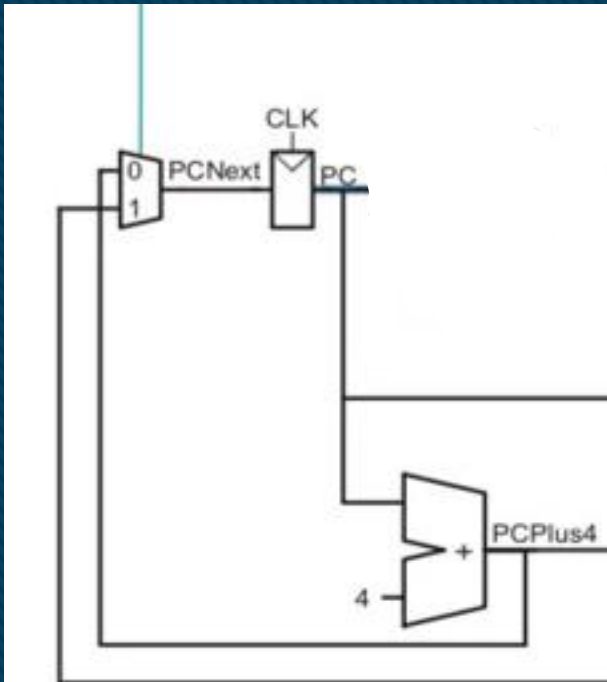
Single Cycle RISC-V

Full Block Diagram



Each Block

Program Counter (PC)



```
`timescale 1ns / 1ps

module PCMUX(
    input wire [31 : 0] pc_plus4,
    input wire [31 : 0] PCTarget,
    input wire pcsrc ,
    output wire [31 : 0] pc_next
);
    assign pc_next = (pcsrc) ? PCTarget : pc_plus4;
endmodule
```

```
module Adder_4 (PC, PCPlus4);

    input  wire [31 : 0] PC;
    output wire [31 : 0] PCPlus4;

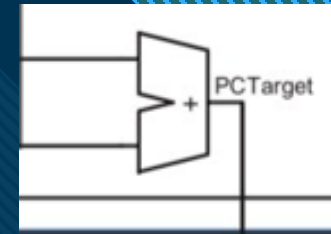
    assign PCPlus4 = PC +4;
endmodule
```

```
module Adder_Target (PC, ImmExt, PCTarget);

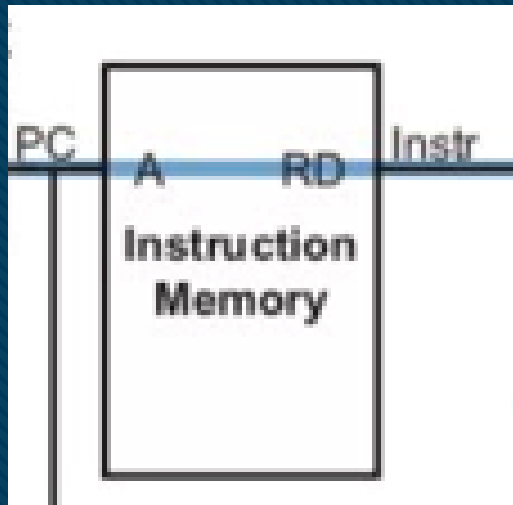
    input  signed [31:0] PC;
    input  signed [31:0] ImmExt;
    output signed [31:0] PCTarget;

    assign PCTarget = PC + ImmExt;

endmodule
```



Instruction Memory



```
timescale 1ns / 1ps

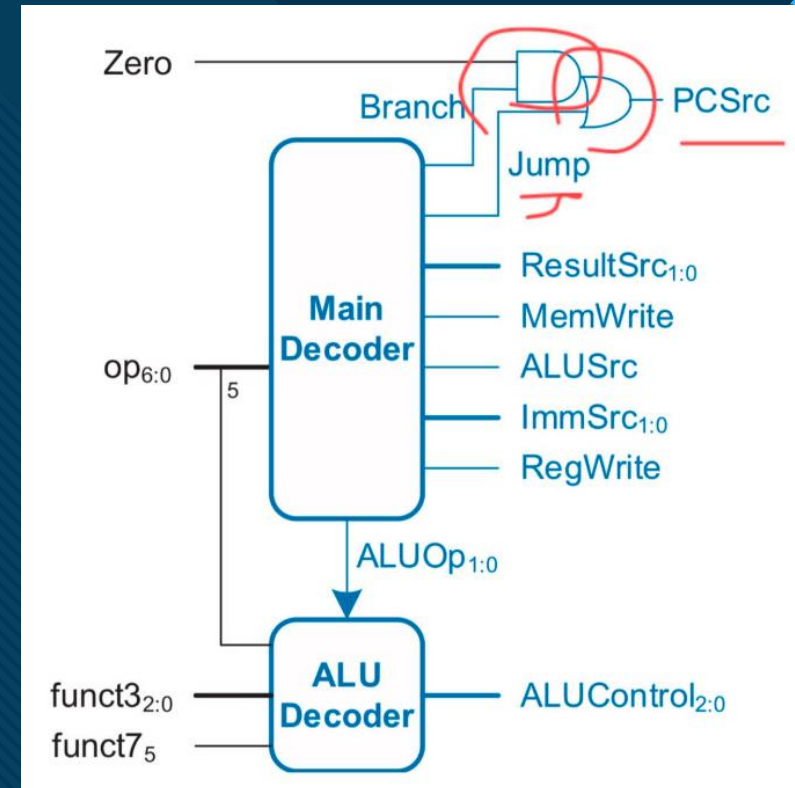
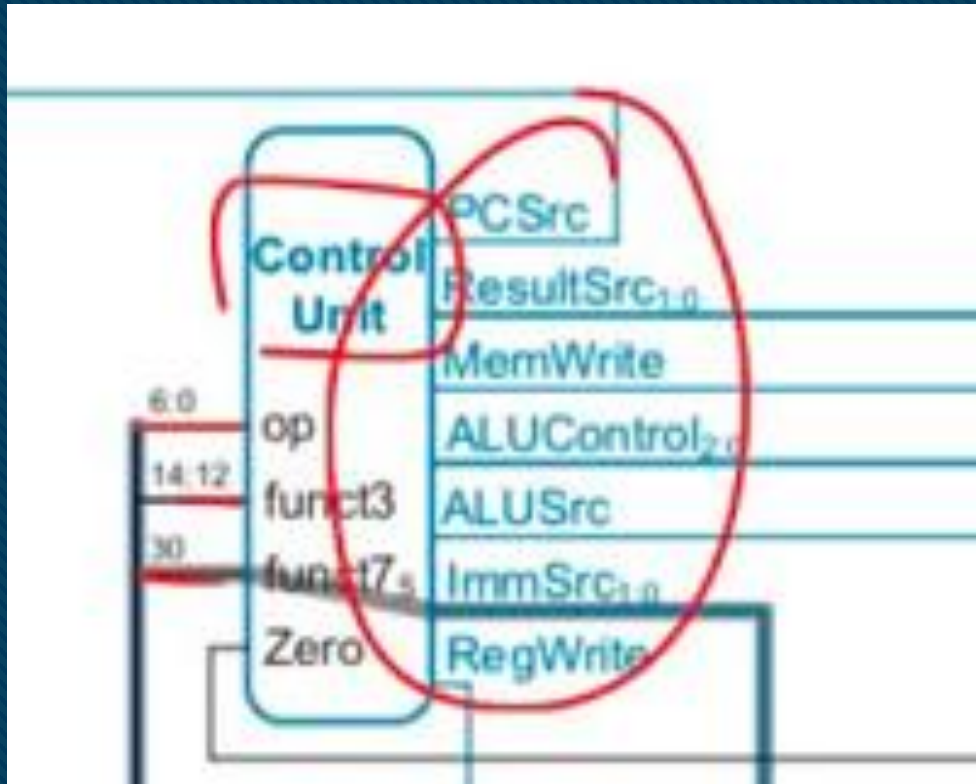
module Instruction_memory(
    input  [31:0] A,
    output [31:0] RD
);

    reg [31:0] mem [0:31];
    integer i;

    initial
    begin
        for (i = 0; i < 32; i = i + 1) begin
            mem[i] = 32'h00000000;
        end
        $readmemh("inst_mem.txt", mem);
    end

    assign RD = mem [A[31:2]];
endmodule
```

Control Unit



Cont: Control Unit

Main Decoder

```
`timescale 1ns / 1ps
module Main_Decoder(Op, Jump, Branch, MemWrite, RegWrite, AluSrc, ImmSrc, ResultSrc, AluOp);
    input wire [6:0] Op;
    output wire Jump, Branch, MemWrite, RegWrite, AluSrc;
    output wire [1:0] ImmSrc, ResultSrc, AluOp;

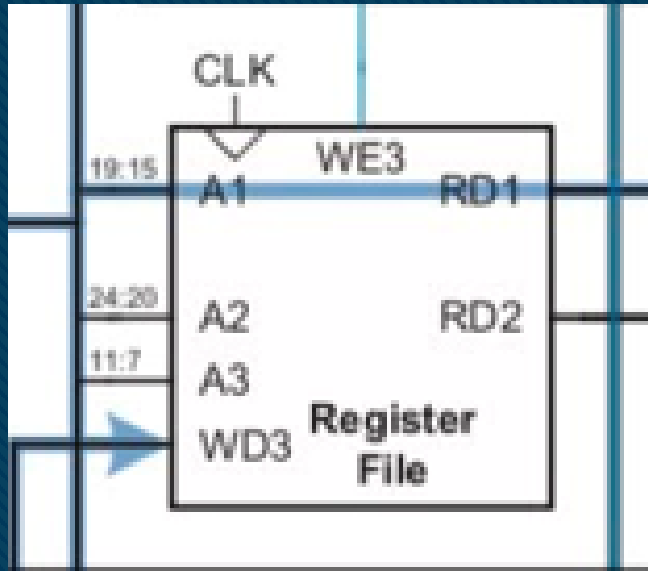
    reg [10:0] controls;
    assign {RegWrite, ImmSrc, AluSrc, MemWrite, ResultSrc, Branch, AluOp, Jump} = controls;

    always @(*)
    begin
        case (Op)
            7'b0000011: controls = 11'b10010010000; // Load
            7'b0100011: controls = 11'b00111xx0000; // Store
            7'b0110011: controls = 11'b1xx00000100; // R-type
            7'b1100011: controls = 11'b01000xx1010; // Branch
            7'b0010011: controls = 11'b10010000100; // I-type
            7'b1101111: controls = 11'b111x0100xx1; // Jump
            default: controls = 11'b00000000000;
        endcase
    end
endmodule
```

ALU Decoder

```
`timescale 1ns / 1ps
module ALU_Decoder (opb5, funct3, funct7b5, ALUOp, ALUControl);
    input wire opb5;
    input wire [2:0] funct3;
    input wire funct7b5;
    input wire [1:0] ALUOp;
    output reg [2:0] ALUControl;
    wire RtypeSub;
    assign RtypeSub = funct7b5 & opb5;
    always @(*) begin
        case (ALUOp)
            2'b00:
                ALUControl = 3'b000;
            2'b01:
                ALUControl = 3'b001;
            2'b10:
                begin
                    case (funct3)
                        3'b000: ALUControl = (RtypeSub) ? 3'b001 : 3'b000;
                        3'b001: ALUControl = 3'b110;
                        3'b010: ALUControl = 3'b101;
                        3'b100: ALUControl = 3'b100;
                        3'b101: ALUControl = 3'b111;
                        3'b110: ALUControl = 3'b011;
                        3'b111: ALUControl = 3'b010;
                        default: ALUControl = 3'bxxx;
                    endcase
                end
            default:
                ALUControl = 3'bxxx;
        endcase
    end
endmodule
```


Register File



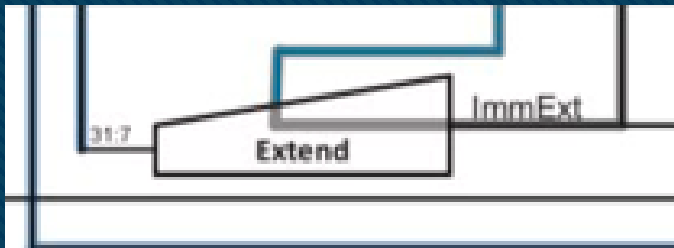
```
`timescale 1ns / 1ps
module Register_File(
    input clk, WE3,
    input [4:0] A1, A2, A3,
    input [31:0] WD3,
    output reg [31:0] RD1, RD2
);

    reg [31:0] RF [0:31];
    integer i;
    initial
    begin
        for (i = 0; i < 32; i = i + 1) begin
            RF[i] = 32'b0;
        end
    end

    always @(posedge clk) begin
        if (WE3 && A3 != 5'b00000)
            RF[A3] <= WD3;
        end

    always @(*) begin
        RD1 = RF[A1];
        RD2 = RF[A2];
    end
endmodule
```


Sign Extension

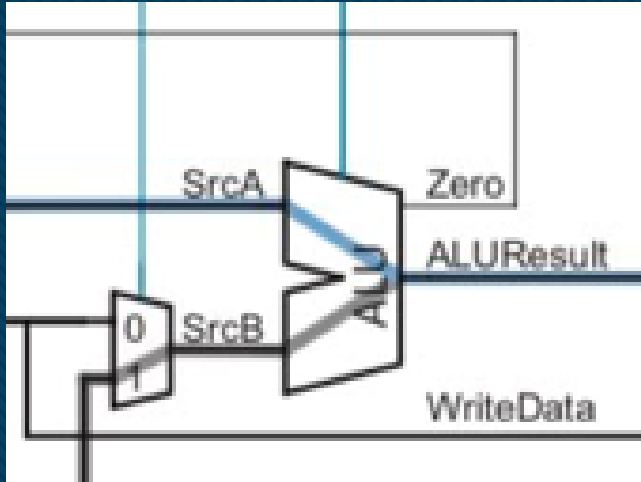


```
`timescale 1ns / 1ps
module Extend_Unit(instr, immsrc, immext);

    input wire [31:7] instr;
    input wire [1:0] immsrc;
    output reg [31:0] immext;

    always @(*)
    begin
        case(immsrc)
            2'b00: immext = {{20{instr[31]}}, instr[31:20]};
            2'b01: immext = {{20{instr[31]}}, instr[31:25], instr[11:7]};
            2'b10: immext = {{20{instr[31]}}, instr[7], instr[30:25], instr[11:8], 1'b0};
            2'b11: immext = {{12{instr[31]}}, instr[19:12], instr[20], instr[30:21], 1'b0};
            default: immext = 32'bx;
        endcase
    end
endmodule
```

ALU and ALU input mux

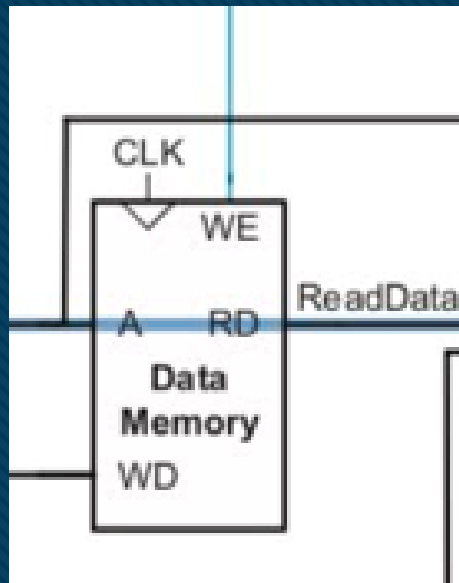


```
`timescale 1ns / 1ps
module ALUINPUTMUX(
    input wire [31 : 0] reg_data,
    input wire [31 : 0] ImmExx ,
    input wire alu_src ,
    output wire [31:0] SrcB
);
assign SrcB = (alu_src) ? ImmExx : reg_data ;
endmodule
```

```
`timescale 1ns / 1ps
module ALU (SrcA, SrcB, ALUControl, ALUResult, Zero);
    input signed [31:0] SrcA;
    input signed [31:0] SrcB;
    input [2:0] ALUControl;
    output reg signed [31:0] ALUResult;
    output reg Zero;

    always @(*) begin
        Zero = 0;
        case (ALUControl)
            3'b000: ALUResult = SrcA + SrcB;
            3'b001:
                begin
                    ALUResult = SrcA - SrcB;
                    Zero = (ALUResult == 32'b0);
                end
            3'b010: ALUResult = SrcA & SrcB;
            3'b011: ALUResult = SrcA | SrcB;
            3'b100: ALUResult = SrcA ^ SrcB;
            3'b101: ALUResult = (SrcA < SrcB) ? 1 : 0;
            3'b110: ALUResult = SrcA << SrcB;
            3'b111: ALUResult = SrcA >> SrcB;
            default: ALUResult = 32'bx;
        endcase
    end
endmodule
```

Data Memory



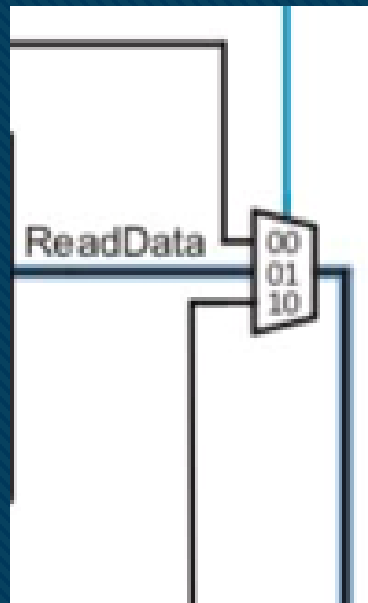
```
`timescale 1ns / 1ps
module Data_Memory(clk, we, a, wd, rd);

    input wire clk;
    input wire we;
    input wire [31:0] a;
    input wire [31:0] wd;
    output reg [31:0] rd;

    reg [31:0] RAM [63:0];

    always @(*) begin
        rd = RAM[a[31:2]];
    end
    always @(posedge clk) begin
        if (we) begin
            RAM[a[31:2]] <= wd;
        end
    end
endmodule
```

Jump/Load mux



```
`timescale 1ns / 1ps
module J_LMUX(
    input wire [31 : 0] ALUResult,
    input wire [31 : 0] ReadData,
    input wire [31 : 0] PCPlus4,
    input wire [1 : 0] ResultSrc,
    output reg [31 : 0] Result
);

always @(*) begin
    case (ResultSrc)
        2'b00: Result = ALUResult;
        2'b01: Result = ReadData;
        2'b10: Result = PCPlus4;
        default: Result = 32'b0;
    endcase
end
endmodule
```

Integration

Data Path

```
`timescale 1ns / 1ps
module Data_path(
    input Clk, Reset, Pcsrc,
    input AluSrc, RegWrite,
    input [1:0] ResultSrc, ImmSrc,
    input [2:0] AluControl,
    input [31:0] Instr,
    input [31:0] ReadData,
    output Zero,
    output [31:0] Pc, DataAdr, WriteData
);

    wire [31:0] PCNext, RD1, RD2,
               PCPlus4, Result,
               ImmExt, PCTarget, SrcB;

    Program_Counter PC(.clk(Clk), .rst(Reset), .PCNext(PCNext), .PC(Pc));

    Adder_4 AD4 (.PC(Pc), .PCPlus4(PCPlus4));

    ALU alu (.SrcA(RD1), .SrcB(SrcB), .ALUControl(AluControl), .ALUResult(DataAdr), .Zero(Zero));

    J_LMUX Jmux(.ALUResult(DataAdr), .ReadData(ReadData), .PCPlus4(PCPlus4), .ResultSrc(ResultSrc), .Result(Result));

    Register_File RF (.clk(Clk), .WE3(RegWrite), .A1(Instr[19:15]), .A2(Instr[24:20]), .A3(Instr[11:7]), .WD3(Result),
                     .RD1(RD1), .RD2(RD2));

    assign WriteData = RD2;

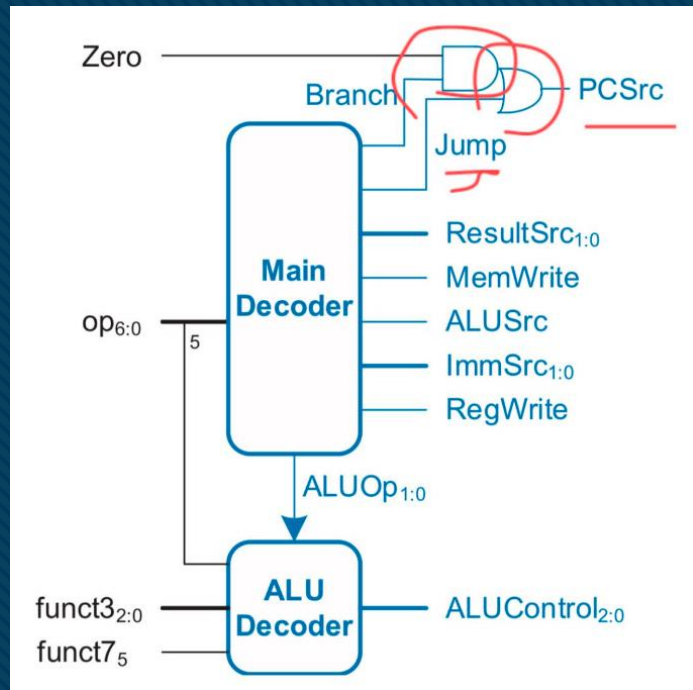
    Extend_Unit EXTEND(.instr(Instr[31:7]), .immsrc(ImmSrc), .immext(ImmExt));

    Adder_Target ADDTARGET (.PC(Pc), .ImmExt(ImmExt), .PCTarget(PCTarget));

    PCMUX pcmux (.pc_plus4(PCPlus4), .PCTarget(PCTarget), .pcsrc(Pcsrc), .pc_next(PCNext));

    ALUINPUTMUX ALUMUX (.reg_data(RD2), .ImmExx(ImmExt), .alu_src(AluSrc), .SrcB(SrcB));
endmodule
```

Control Unit



```
`timescale 1ns / 1ps
module Controller (Zero, Instr, MemWrite, RegWrite, ImmSrc, ALUSrc, ALUControl, ResultSrc, PcSrc);

    input wire Zero;
    input wire [31:0] Instr;
    output wire MemWrite;
    output wire RegWrite;
    output wire [1:0] ImmSrc;
    output wire ALUSrc;
    output wire [2:0] ALUControl;
    output wire [1:0] ResultSrc;
    output wire PcSrc;
    wire Jump, Branch;
    wire [1:0] AluOp;

    Main_Decoder MD(
        .Op(Instr[6:0]),
        .Jump(Jump),
        .Branch(Branch),
        .MemWrite(MemWrite),
        .RegWrite(RegWrite),
        .AluSrc(ALUSrc),
        .ImmSrc(ImmSrc),
        .ResultSrc(ResultSrc),
        .AluOp(AluOp)
    );

    assign PcSrc = Jump | ((Zero ^ (Instr[12])) & Branch);

    ALU_Decoder AD (
        .opb5(Instr[5]),
        .funct3(Instr[14:12]),
        .funct7b5(Instr[30]),
        .ALUOp(AluOp),
        .ALUControl(ALUControl)
    );

endmodule
```


Processor

```
`timescale 1ns / 1ps
module RV32I_Processor(Clk, Reset, Instr, ReadData, Pc, DataAdr, WriteData, MemWrite);

    input  wire Clk;
    input  wire Reset;
    input  wire [31:0] Instr, ReadData;
    output wire MemWrite;
    output wire [31:0] Pc;
    output wire [31:0] DataAdr;
    output wire [31:0] WriteData;

    wire Zero, RegWrite, ALUSrc, PcSrc;
    wire [1:0] ImmSrc, ResultSrc;
    wire [2:0] ALUControl;

    Controller Cont ( .Zero(Zero), .Instr(Instr), .MemWrite(MemWrite), .RegWrite(RegWrite), .ImmSrc(ImmSrc), .ALUSrc(ALUSrc),
        .ALUControl(ALUControl), .ResultSrc(ResultSrc), .PcSrc(PcSrc));

    Data_path DP ( .Clk(Clk), .Reset(Reset), .Pcsrc(PcSrc), .AluSrc(ALUSrc), .RegWrite(RegWrite), .ResultSrc(ResultSrc),
        .ImmSrc(ImmSrc), .AluControl(ALUControl), .Instr(Instr), .ReadData(ReadData),
        .Zero(Zero), .Pc(Pc), .DataAdr(DataAdr), .WriteData(WriteData));

endmodule
```

Single Cycle 32-bit RISC-V

```
`timescale 1ns / 1ps
module SC_32_RISC_V(

    input Clk,Reset
);

    wire [31:0] Instr,ReadData,Pc,
               DataAdr,WriteData;
    wire MemWrite;
    RV32I_Processor Processor (.Clk(Clk), .Reset(Reset), .Instr(Instr), .MemWrite(MemWrite),
                              .ReadData(ReadData), .Pc(Pc), .DataAdr(DataAdr), .WriteData(WriteData)
                              );

    Instruction_memory Instr_mem ( .A(Pc), .RD(Instr));

    Data_Memory data_mem ( .clk(Clk), .we(MemWrite), .a(DataAdr), .wd(WriteData), .rd(ReadData));

endmodule
```

Test Bench

The background is a dark blue gradient. A diagonal line runs from the bottom-left towards the top-right. To the left of this line is a lighter blue area. To the right is the dark blue area. A thin, hatched blue line follows the diagonal line, separating the two blue areas.

Thank You